

Flujos del Sistema y Casos de Prueba

Ejecutados y Validados

Microservice Code Studio — AgentIA

Equipo de Ingeniería — Aseguramiento de Calidad (QE)

2 de octubre de 2026

Objeto de este documento. Documentar los **flujos** del sistema, explicar **cómo se probaron** y **exponer los casos de prueba ejecutados y validados**, con la evidencia que respalda cada cifra.

Toda cifra de la Sección 8 está **generada** a partir de los informes JUnit que producen las propias suites, no escrita a mano: si una prueba cambia de resultado, la tabla cambia. El detalle caso por caso está en docs/entrega_qe/matriz_casos_de_prueba.csv.

Contents

1	Resumen ejecutivo	3
2	Alcance y método	3
2.1	Qué se probó	3
2.2	Cómo se probó: el método de cinco pasos	3
2.3	Evidencia y reproducibilidad	4
3	Los flujos del sistema	4
3.1	F0 — Autenticación y sesión de operador	4
3.2	F1 — Ingesta y validación de especificación (blueprints)	4
3.3	F2 — Requisitos a Historias BDD	5
3.4	F3 — Diseño arquitectónico	5
3.5	F4 — Modelos de dominio y esquema SQL	5
3.6	F5 — Generación de código y síntesis de pruebas	5
3.7	F6 — Sandbox hermético y auto-reparación acotada	6
3.8	F7 — Auditoría de seguridad y Quality Gate	6
3.9	F8 — DevOps: manifiestos, contenedores y despliegue local	6
3.10	F9 — Exportación, artefactos y publicación Git	6
3.11	F10 — Orquestación Auto-Pilot y ciclo de vida de la sesión	6
3.12	F11 — Trazabilidad de costos y telemetría	7
3.13	F12 — Playground de API en vivo	7
3.14	FT — Flujos transversales de la interfaz	7
4	Tipos de prueba y estrategia	7
5	Casos de prueba ejecutados y validados	8
6	Trazabilidad	9
7	Defectos encontrados y corregidos	9
8	Resultados de ejecución	10
9	Limitaciones	12

10 Anexo A — Fragmentos de código representativos	12
11 Anexo B — Paquete de entrega	13

1 Resumen ejecutivo

El sistema se organiza en **14 flujos** funcionales sobre **62 endpoints** REST. Esta campaña documenta cada flujo, lo prueba y expone el resultado de cada caso.

Métrica	Valor
Flujos documentados	14
Endpoints REST cubiertos por la documentación	62
Casos de prueba ejecutados	1194
Backend (pytest)	1126
Frontend (Vitest + MSW)	68
Archivos con casos de prueba	106
Defectos encontrados y corregidos	9

Las cifras exactas de resultado — cuántos casos pasan, cuántos no, y por qué — se generan en la Sección 8. Se actualizan solas al reejecutar.

Las cifras por módulo de la tabla anterior son el resultado medido al cierre de la campaña. El código ha cambiado desde entonces en paralelo: la medición actual, regenerada con el mismo comando, está en la Sección 8.

Lo que el lector debe llevarse

1. **Los flujos están documentados uno a uno** (Sección 3): qué los dispara, qué producen, y qué casos de prueba los validan.
2. **Las pruebas se ejecutan, no se declaran.** Cada caso lleva su resultado real tomado del informe de la suite (Sección 5).
3. **Nueve defectos reales aparecieron durante la campaña** y están corregidos, cada uno con la prueba de regresión que impide que vuelva (Sección 7).
4. **Hay límites, y se declaran** (Sección 9). El más importante: cobertura no es corrección, y la plataforma verifica el código que ella misma genera.

2 Alcance y método

2.1 Qué se probó

La superficie completa del producto: los 13 flujos funcionales, desde la autenticación del operador hasta la publicación del microservicio generado, incluyendo la interfaz web.

2.2 Cómo se probó: el método de cinco pasos

1. **Medir antes de tocar nada.** La cobertura de partida se registra sobre el código sin modificar. Sin línea base no hay mejora demostrable.
2. **Priorizar por riesgo, no por cobertura.** Se elige primero el módulo cuya decisión duele más si es incorrecta, aunque su porcentaje sea alto.
3. **Enunciar la afirmación del módulo.** Antes de escribir una prueba se escribe la frase que el módulo promete: “*HEALTHY sólo es honesto si un /actuator/health real respondió UP*”. La prueba verifica esa frase, no una función.
4. **Sustituir la frontera, nunca el sujeto.** Se falsean las dependencias que no son nuestras (Docker, red, hilos, LLM, reloj). Jamás se falsea lo que se está probando.

5. **Ver la prueba fallar.** Una prueba que nunca se vio en rojo no es evidencia de nada: puede estar pasando por la razón equivocada.

2.3 Evidencia y reproducibilidad

Todo el material está versionado en `reports/qe/entrega/`: salida completa de cada suite, cobertura por archivo en formato consumible, e informes JUnit de los que se deriva la matriz de casos. La Sección 8 contiene los comandos exactos.

3 Los flujos del sistema

Cada flujo se documenta con la misma ficha: **propósito**, **entrada**, **pasos**, **artefactos** que produce, y **validación** (los casos de prueba que lo cubren). El identificador F# se usa en la matriz de casos.

3.1 F0 — Autenticación y sesión de operador

Propósito. Que sólo un operador con sesión válida pueda usar el estudio. **Entrada.** Credenciales configuradas, o acceso local automático. **Salida.** Cookie de sesión `agentia_session`. **Endpoints.** 4.

Paso	Descripción
Entrada local	POST <code>/auth/mvp</code> . Sólo se concede a llamadas desde <i>loopback</i> y con <code>STUDIO_AUTO_LOGIN</code> activo. Es la comodidad de “esta máquina”, no de “cualquiera”.
Entrada con credencial	POST <code>/auth/login</code> . Exige <code>STUDIO_USER_EMAIL</code> y un token de al menos 16 caracteres; si no están configurados responde 503, es decir falla cerrado en lugar de aceptar.
Verificación	GET <code>/auth/session</code> devuelve el operador, o 401.
Protección	Un <i>middleware</i> responde 401 a toda ruta bajo <code>/api/v1</code> sin cookie válida. Las rutas de autenticación y <code>/healthz</code> son públicas.
Salida	POST <code>/auth/logout</code> revoca la sesión en el servidor y borra la cookie.

Validación. `test_qe_auth_flow.py` (27 casos): cobertura de la guarda sobre el prefijo completo, imposibilidad de saltarla con cabeceras, rechazo a llamadas remotas, fallo cerrado con credencial corta o ausente, comparación de correo insensible a mayúsculas, límite de intentos (429) y su caducidad, endurecimiento de la cookie (`HttpOnly`, `SameSite=strict`, `Path=/api/v1`), token no almacenado en claro, rechazo de token falsificado, y revocación efectiva tras `logout`.

3.2 F1 — Ingesta y validación de especificación (blueprints)

Propósito. Aceptar una especificación formal ya existente y validarla antes de generar nada. **Entrada.** JSON de blueprint o documento subido. **Salida.** Especificación validada con identificador. **Endpoints.** 3.

Pasos. POST `/specifications` valida el blueprint contra el esquema (entidades, atributos, historias, escenarios) y lo persiste; POST `/specifications/upload` hace lo propio con un documento; GET `/specifications/{id}` lo recupera.

Validación. `test_routes_spec.py`, `test_qe_specification_guard.py`, `test_qe_specification_persistence.py`, `test_entry_point_parity.py`. Se fija que un blueprint malformado no se persiste,

que la especificación sobrevive a la sesión, y que los dos puntos de entrada (documento y formulario) producen el mismo resultado.

3.3 F2 — Requisitos a Historias BDD

Propósito. Convertir una descripción de negocio en historias de usuario con criterios de aceptación *Given/When/Then*. **Salida.** Historias BDD + entidades de dominio inferidas. **Endpoints.** 4.

Pasos. `POST /requirements/transform` descompone el texto; `POST /requirements/refine` lo ajusta; `POST /requirements/sessions/{id}/save` y `GET /requirements/sessions/{id}` persisten y leen el borrador.

Validación. `test_routes_requirements.py`, `test_requirements_service.py`, `test_qe_api_surface.py`, `test_qe_structured_output_fallback.py`. Se fija, entre otros contratos, que cada cláusula *given/when/then* exige al menos 3 caracteres, y que el modo sin conexión **declara** que su descomposición es una plantilla fija en lugar de presentarla como respuesta al prompt.

3.4 F3 — Diseño arquitectónico

Propósito. Producir el blueprint en 4 capas (controlador, servicio, repositorio, entidad) y el catálogo de DTOs. **Salida.** `architecture.json`, `architecture.md`, diagrama Mermaid. **Endpoints.** 3.

Validación. `test_routes_architecture.py`, `test_architecture_service.py`, `test_routes_artifact.py`. Se fija la separación estricta de capas, que los DTOs son *records* inmutables (Principio II) y que la serialización Mermaid es estable.

3.5 F4 — Modelos de dominio y esquema SQL

Propósito. Derivar entidades JPA y el DDL relacional *del mismo blueprint*. **Salida.** `schema.sql`, `data.sql`, entidades Jakarta. **Endpoints.** 3.

Validación. `test_routes_models_sql.py`, `test_model_sql_service.py`, `test_qe_schema_consistency.py`, `test_qe_slice_test_emission.py`. El caso central es la **coherencia esquema–entidad**: la regla `SCHEMA_ENTITY_MISMATCH` compara dos artefactos generados entre sí, porque un esquema que crea la tabla `items` junto a una entidad que mapea `orders` hace que el servicio no arranque contra su propia base de datos.

3.6 F5 — Generación de código y síntesis de pruebas

Propósito. Sintetizar el microservicio Spring Boot 3 / Java 21 y sus pruebas. **Salida.** Proyecto Maven, fuentes Java, suites Mockito. **Endpoints.** 3 de pruebas + el paso correspondiente del orquestador.

Validación. `test_routes_tests.py`, `test_test_analysis_service.py`, `test_code_generation_module.py`, `test_qe_generated_code_fixes.py`. Se fija el Principio I (Java 21, Spring Boot 3.x), el Principio II (records para DTOs) y el Principio III (manejador centralizado de errores).

3.7 F6 — Sandbox hermético y auto-reparación acotada

Propósito. Compilar y ejecutar las pruebas *sin red*, diagnosticar los fallos y reparar hasta 3 veces. **Salida.** Veredicto de verificación y, si procede, parche. **Endpoints.** 5.

Pasos. POST /tests/analyze extrae diagnósticos estructurados (archivo, línea, tipo, esperado vs. obtenido); POST /tests/repair aplica un parche quirúrgico y reejecuta; GET /sessions/{id}/repairs expone el historial; POST /sessions/{id}/manual-repair permite la intervención humana.

Validación. test_sandbox_verifier_honesty.py, test_qe_environment_repair.py, test_repair_parser.py, test_sandbox_mount_suffix.py, test_docker_runner.py. El contrato de honestidad es el eje: **una verificación que no se ejecutó no es una sesión verificada**. El sistema distingue tres estados que nunca deben confundirse: *verificado*, *verificado con hallazgos* y *no evaluable*.

3.8 F7 — Auditoría de seguridad y Quality Gate

Propósito. Auditar el código generado (SAST, secretos, CVE) y decidir si puede salir. **Salida.** Veredicto del Quality Gate y hallazgos. **Endpoints.** 5.

Validación. test_routes_security.py, test_security_service.py, test_qe_injection_guard.py. Este último ataca el sistema con rutas y etiquetas hostiles, y comprueba que ninguna se convierte en ejecución o en fuga de información.

3.9 F8 — DevOps: manifiestos, contenedores y despliegue local

Propósito. Generar Dockerfile, Compose, CI/CD y Kubernetes; desplegar localmente y comprobar salud. **Salida.** Manifiestos y estado del contenedor. **Endpoints.** 9.

Validación. test_routes_devops.py, test_devops_service.py, test_qe_deploy_docker.py, test_qe_deployment_identity.py. El contrato central: docker compose up con código de salida 0 significa “los contenedores arrancaron”, **nunca** “el servicio funciona”. Sólo un /actuator/health real que responda UP autoriza el estado HEALTHY.

3.10 F9 — Exportación, artefactos y publicación Git

Propósito. Entregar el proyecto (ZIP) y publicarlo en una rama Git. **Endpoints.** 5.

Validación. test_qe_publish_git.py, test_git_service.py, test_routes_publish.py, test_artifact_retrieval.py, test_export_service.py. Principio VI en primer plano: el token de acceso se usa y se **borra** en un bloque finally, de modo que no sobrevive ni en el caso de publicación fallida, y nunca aparece en un mensaje de error.

3.11 F10 — Orquestación Auto-Pilot y ciclo de vida de la sesión

Propósito. Ejecutar la cadena completa sin intervención, con pausa, reanudación y cancelación, y exponer el avance en vivo. **Endpoints.** 13 + 6 de sesiones. **Es el flujo de mayor riesgo:** nadie lo vigila mientras corre.

Pasos. POST /orchestrator/pipeline/run arranca; {session_id}/pause, /resume y /cancel gobiernan la ejecución; GET /{session_id}/events emite el avance por SSE; /lifecycle y /overview devuelven el estado de las 7 fases; /transition e

/invalidate mueven la máquina de estados; POST /sessions y /sessions/quick-start crean sesiones y GET /sessions/{id}/stream emite los eventos de sesión.

Validación. test_qe_pipeline_autopilot.py, test_pipeline_runner.py, test_routes_orchestrator.py, test_qe_lifecycle_rules.py, test_qe_lifecycle_artifacts.py, test_qe_outdated_phases.py, test_lifecycle_service.py, test_qe_session_surface.py, test_sessions_api.py, test_routes_session.py, test_e2e_flow.py. Se fija que la pausa surte efecto en la *siguiente* frontera de paso, que cancelar no se confunde con pausar, que el camino de agotamiento escribe estado y diagnóstico juntos, y que un fallo dentro del grafo termina la sesión en lugar de dejarla en RUNNING para siempre.

3.12 F11 — Trazabilidad de costos y telemetría

Propósito. Registrar el costo real de cada llamada al modelo y poder auditarlo. **Salida.** Registros de costo por llamada y por sesión.

Validación. test_cost_recording.py, test_cost_pricing.py, test_cost_report.py, test_cost_seam.py, test_cost_mlflow_mirror.py, test_qe_mlflow_tracing.py. Se fija que un uso desconocido se registra como *no valorado*, nunca como cero; y que el espejo de telemetría es de mejor esfuerzo y **no puede bloquear** la ruta crítica.

3.13 F12 — Playground de API en vivo

Propósito. Permitir invocar el microservicio desplegado desde la interfaz. **Endpoints.** 2.

Validación. test_qe_playground_proxy.py, test_qe_playground_targeting.py: sólo se permite apuntar al servicio propio.

3.14 FT — Flujos transversales de la interfaz

Propósito. La superficie web: inicio de sesión, navegación entre pestañas, tema, exploradores de artefactos y los flujos de usuario de extremo a extremo. **Validación.** workflow_journeys.test.tsx (9 casos), app_integration.test.tsx, layout.test.tsx, auth.test.tsx, llm.test.tsx, theme.test.tsx, services.test.ts, views_*.test.tsx.

Los flujos de usuario se prueban contra un backend simulado en la frontera HTTP (MSW), no contra la capa de servicios falseada. Es la diferencia entre comprobar que un componente dibuja algo y comprobar que la aplicación *hace la petición correcta*: URL, método, parámetros, cuerpo y cabeceras reales.

4 Tipos de prueba y estrategia

Tipo	Qué verifica	Dónde se aplica
Unitaria	Una función o clase, con sus colaboradores falseados	Reglas de fase, precios de costo, parser de reparación
Integración de módulo	Varios colaboradores reales en conjunto	Pipeline Auto-Pilot completo en un solo caso
De contrato	La interfaz entre dos componentes	Los 9 flujos de usuario del frontend; esquemas de petición
De extremo a extremo	Un camino de usuario completo	test_e2e_flow.py; inicio rápido → pestaña → smoke test

Tipo	Qué verifica	Dónde se aplica
De regresión	Que un defecto concreto no vuelva	Los 6 defectos de la Sección 7
De caracterización	Fijar el comportamiento actual para que cambiarlo sea deliberado	Modo sin conexión que se declara plantilla
Negativa / inyección de fallo	Que el sistema falle bien	D-04, D-05, smoke test fallido, inyección hostil
De hermetismo	Que la suite no toque la red	Fixture <code>no_outbound_telemetry</code>

Fuera de alcance, declarado: no hay pruebas de carga ni de rendimiento, ni de accesibilidad, ni de navegador real, ni mutación, ni *fuzzing*.

Sustitución de fronteras

Regla aplicada en todo el trabajo: **se falsea la frontera que no es nuestra; jamás el sujeto bajo prueba.**

Frontera	Sustituto	Motivo
<code>subprocess.Popen/run</code>	proceso falso guionado	sin Docker
<code>requests</code>	respuesta falsa o excepción	sin red
<code>threading.Thread</code>	ejecución en línea	determinismo, sin esperas
Grafo LangGraph	grafo guionado	sin LLM
Auditoría SAST	auditoría falsa	sin motor externo
HTTP (frontend)	backend simulado con MSW	flujo real sin servidor vivo

5 Casos de prueba ejecutados y validados

Qué se ejecutó

Se ejecutaron las dos suites completas sobre el estado actual del código. Cada caso lleva identificador estable por flujo (F6-014, FT-233), resultado, duración y, si no pasó, el motivo que reportó el propio ejecutor.

- **Matriz completa:** `docs/entrega_qe/matriz_casos_de_prueba.csv` (una fila por caso; columnas: id, flujo, suite, archivo, caso, resultado, segundos, motivo).
- **Resumen por flujo:** `docs/entrega_qe/resumen_ejecucion.json`.
- **Evidencia cruda:** `reports/qe/entrega/` — salida de ambas suites e informes JUnit de origen.

Cómo se lee la validación

Un caso se considera **validado** cuando la suite lo reporta como **PASS** en la ejecución más reciente. Un caso en **FAIL** **no se oculta:** aparece en la tabla de casos no superados, con el motivo, y se explica en las limitaciones. Un caso **SKIP** está omitido de forma deliberada y también se declara.

Nota de honestidad sobre estas cifras. Las pruebas no demuestran que el sistema sea correcto. Demuestran que *estos* casos, ejecutados sobre *este* código, producen *este* resultado. La cobertura mide qué se ejecutó, no qué es verdad.

6 Trazabilidad

Cada principio de la constitución del proyecto y cada requisito funcional relevante tiene al menos una prueba que lo fija. Sin esto, una prueba es una afición; con esto, es evidencia.

Principio / requisito	Prueba que lo fija
I — Java 21 / Spring Boot 3.x	<code>test_code_generation_module.py</code> , <code>test_qe_generated_code_fixes.py</code>
II — DTOs como <i>records</i> inmutables	<code>test_architecture_service.py</code> , análisis de cumplimiento
III — Manejo centralizado de errores	<code>test_qe_api_surface.py</code> , <code>test_response_contract.py</code>
IV — Compilación hermética y sin red	<code>test_sandbox_verifier_honesty.py</code> , <code>test_docker_runner.py</code> , <code>no_outbound_telemetry</code>
V — Auto-reparación acotada a 3 intentos	<code>test_qe_lifecycle_rules.py</code> , <code>test_qe_pipeline_autopilot.py</code>
VI — Cero secretos persistentes	<code>test_qe_publish_git.py</code> , <code>test_cost_recording.py</code> , <code>workflow_journeys.test.tsx</code>
Honestidad de la verificación (012)	<code>test_sandbox_verifier_honesty.py</code> , <code>test_qe_deploy_docker.py</code> , <code>workflow_journeys.test.tsx</code>
Diagnóstico en todo camino terminal (015)	<code>test_diagnostic_record.py</code> , <code>test_qe_pipeline_autopilot.py</code>
Aislamiento de sesión (autenticación)	<code>test_qe_auth_flow.py</code>
Trazabilidad de costos (013)	<code>test_cost_*.py</code> , <code>test_qe_mlflow_tracing.py</code>

7 Defectos encontrados y corregidos

Nueve defectos reales. **Ninguno se descubrió leyendo el código:** todos aparecieron porque una prueba falló, se colgó, o ejecutó por primera vez una rama que nunca se había ejecutado.

ID	Defecto	Severidad	Causa raíz
D-03	El espejo de telemetría “de mejor esfuerzo” bloqueaba la ruta crítica y colgaba la suite	Crítica	Los valores por defecto de la librería eran 120 s de espera con 7 reintentos; el llamador envuelve <i>cada</i> llamada al modelo
D-05	El respaldo de síntesis del esquema llamaba a un método inexistente	Alta	Función de módulo invocada como método de un objeto; <code>open(..., "w")</code> trunca antes de evaluar su argumento, así que dejaba el archivo vacío
D-04	El fallo más ruidoso escapaba del manejador de errores	Alta	El <code>try</code> del pipeline empezaba <i>después</i> de la decisión de modo: la excepción moría con el hilo y la sesión quedaba en <code>RUNNING</code>
D-02	La razón del bloqueo se calculaba y se descartaba	Alta	Se asignaba una variable local que nadie leía; el objeto de estado se construye desde otra

ID	Defecto	Severidad	Causa raíz
D-06	La autenticación de sesión invalidó 154 pruebas de golpe	Alta	Un control de seguridad nuevo sin prueba propia cambió el contrato de todos los endpoints
D-01	Dos funciones definidas dos veces en el mismo archivo	Media	La segunda definición sepulta a la primera, que queda como código muerto con contrato distinto
D-08	Un refactor posterior revirtió el respaldo del esquema y dejó la importación sin usar	Alta	El respaldo derivado del blueprint se sustituyó por un <code>raise</code> : cualquier fallo de síntesis mataba la sesión, y desaparecía la salvaguarda del defecto D-05
D-09	La suite no era independiente del orden: 52 casos pasaban aislados y fallaban en la ejecución completa	Alta	Un archivo de prueba limpiaba el almacén de sesiones global y dejaba sin validez las cookies de los clientes que los demás archivos crean al importarse
D-07	El parámetro <code>force</code> de <code>run_pipeline</code> dejó de respetarse	Alta	La guarda de concurrencia se cambió a <code>if t and t.is_alive()</code> , de modo que <code>resume_pipeline</code> – que llama con <code>force=True</code> – fallaba en silencio si el trabajador anterior seguía vivo

Cada corrección dejó una **prueba de regresión** que falla si el defecto vuelve. El detalle completo de causa, evidencia y corrección está en el informe técnico `docs/qe_plan_y_resultados.pdf`, que conserva D-01 a D-05 con su traza.

D-06, y por qué importa para este documento. Al añadirse la autenticación de sesión, 154 pruebas existentes fallaron a la vez, todas con 401, ninguna equivocada sobre lo que probaba. Un control de seguridad **sin una sola prueba propia** había cambiado el contrato de los 62 endpoints, y la suite sólo podía reportarlo como un muro de fallos inconexos. Se corrigió en dos partes: un cliente de prueba autenticado que entra por el endpoint real (`tests/auth_client.py`, sin saltarse el control), y 27 casos nuevos que prueban el control mismo (`test_qe_auth_flow.py`).

8 Resultados de ejecución

Resultado	Casos	%
Correctos (PASS)	1191	99.7
Fallidos (FAIL)	0	0.0
Omitidos (SKIP)	3	0.3
Total ejecutado	1194	100,0

Flujo	Nombre	Casos	PASS	FAIL
F0	Autenticación y sesión de operador	28	28	0
F1	Ingesta y validación de especificación (blueprints)	34	34	0
F10	Orquestación Auto-Pilot y ciclo de vida de la sesión	180	180	0
F11	Trazabilidad de costos y telemetría	57	56	0

Flujo	Nombre	Casos	PASS	FAIL
F12	Playground de API en vivo	34	34	0
F2	Requisitos → Historias BDD	13	13	0
F3	Diseño arquitectónico	10	10	0
F4	Modelos de dominio y esquema SQL	39	39	0
F5	Generación de código y síntesis de pruebas	36	36	0
F6	Sandbox hermético y auto-reparación acotada	82	81	0
F7	Auditoría de seguridad y Quality Gate	82	82	0
F8	DevOps: manifiestos, contenedores y despliegue local	68	68	0
F9	Exportación, artefactos y publicación Git	41	41	0
FT	Transversal: interfaz, tema, contratos de servicio	490	489	0

Cobertura del backend

La cobertura se mide sobre `backend/app`. Al cierre de la campaña los cuatro módulos atacados por riesgo quedaron al 100 % de líneas. La tabla siguiente es la medición **actual**, tomada con el mismo comando sobre el código de hoy; las cifras difieren porque el código creció bajo la campaña (una sesión de trabajo concurrente añadió del orden de dos mil sentencias), no porque se haya perdido cobertura.

Módulo	Flujo	Cobertura	Sin cubrir
<code>services/docker_service.py</code>	F8	95.2 %	11
<code>api/routes_session.py</code>	F10	92.0 %	27
<code>services/lifecycle_service.py</code>	F10	97.0 %	9
<code>services/pipeline_runner.py</code>	F10	95.1 %	26
<code>services/auth_service.py</code>	F0	89.0 %	11
Total del backend		86.6 %	1 215

Cobertura no es corrección. Un porcentaje alto significa que esas líneas se ejecutaron al menos una vez bajo prueba. No significa que sean correctas, y no sustituye a la revisión. Las brechas que quedan están listadas en la Sección 9.

Reproducción

```
# Backend: suite completa con cobertura e informe JUnit
.env/bin/python -m pytest -q --cov=backend/app --cov-report=term-missing \
  --junitxml=reports/qe/entrega/junit-backend.xml

# Frontend: suite completa con informe JUnit
cd frontend && npx vitest run --reporter=default --reporter=junit \
  --outputFile.junit=../reports/qe/entrega/junit-frontend.xml

# Regenerar la matriz de casos y las tablas de este informe
.env/bin/python docs/entrega_qe/build_matriz.py
```

9 Limitaciones

1. **La verificación del código generado es autorreferencial.** La plataforma escribe las pruebas Mockito y después las ejecuta. Un `mvn test -o` en verde demuestra que el código hace lo que dicen *las pruebas de la plataforma*, no lo que dicen los escenarios de aceptación del blueprint. Cerrar esa brecha exige pruebas de aceptación HTTP independientes contra el servicio desplegado, y es el siguiente entregable de mayor valor.
2. **No hay navegador real.** Los flujos de interfaz corren en `jsdom` contra un backend simulado. Un fallo que sólo aparezca en un navegador real (CSS, foco, un evento nativo) no se detectaría.
3. **La concurrencia no se prueba: se elimina.** En las pruebas, los hilos se sustituyen por ejecución en línea para garantizar determinismo. Es una decisión de fiabilidad, y deja sin cubrir las condiciones de carrera.
4. **Quedan módulos por debajo del 90 %.** Principalmente `architecture_service.py` y `requirements_service.py`, que producen artefactos entregables: es el mismo perfil que D-05, código que escribe archivos que nadie comprueba hasta el final.
5. **Las omisiones son deliberadas y visibles.** Los casos en `SKIP` aparecen en la matriz, no escondidos.

10 Anexo A — Fragmentos de código representativos

Los fragmentos completos, comentados, están en `docs/entrega_qe/anexo_snippets.md`. Aquí, la anatomía de un caso:

```
def test_a_container_that_never_becomes_healthy_is_failed_not_healthy(
    monkeypatch, tmp_path
):
    """El codigo 0 de compose solo dice que los contenedores arrancaron."""

    # ARRANGE -- se falsea la frontera, nunca el sujeto:
    # el demonio Docker, el proceso, y la sonda de salud.
    monkeypatch.setattr(ds, "check_docker_daemon", lambda: True)
    monkeypatch.setattr(ds.subprocess, "Popen",
                        lambda cmd, **kw: _FakeProcess(0))
    monkeypatch.setattr(ds, "run_smoke_test", lambda *a, **k: SmokeTestResult(
        passed=False, statusCode=503, testUrl="u",
        details="timed out after 20 attempts"))

    # ACT -- una sola llamada.
    session = ds.deploy_local(SESSION_ID, str(tmp_path))

    # ASSERT -- el contrato, no la implementacion.
    assert session.status == DeploymentStatus.FAILED
    assert session.healthStatus is None, (
        "un smoke test fallido no puede declarar estado de salud")
    assert "Smoke test failed" in session.errorMessage
```

Un caso de flujo de usuario, sobre HTTP real simulado:

```
it('informa que el smoke test NO se ejecuto cuando el backend falla', async () => {
    server.use(
        http.get(pathEndsWith('/sessions'), () => HttpResponse.json([SESSION_LIST_ITEM])),
        http.post(apiPath('/devops/[~]/+/smoke-test'),
            () => HttpResponse.json({ message: 'connection refused' }, { status: 502 })),
    );
    render(<App />);
    fireEvent.click(await screen.findByTitle('6. DevOps & Demo'));
    const button = (await screen.findByText('Ejecutar Smoke Test')).closest('button');
    await waitFor(() => expect(button).not.toBeDisabled());
```

```
fireEvent.click(button);  
// La tercera via: "no evaluable" nunca se disfraza de exito.  
await waitFor() =>  
  expect(screen.getByText(/Smoke test NO ejecutado/i)).toBeInTheDocument();  
  expect(screen.queryByText(/Endpoint de salud verificado/i)).not.toBeInTheDocument();  
});
```

11 Anexo B — Paquete de entrega

Archivo	Contenido
informe_qe.pdf	Este documento
anexo_snippets.md	Fragmentos de código comentados, por tipo de prueba
plantilla_presentacion.pptx	Plantilla de diapositivas para la exposición
matriz_casos_de_prueba.csv	Un caso por fila: id, flujo, resultado, motivo
resumen_ejecucion.json	Totales por flujo y por resultado
reports/qe/entrega/	Evidencia cruda: salidas e informes JUnit